Bootstrap

Jyotishka Datta

Virginia Tech

2021/06/21 (updated: 2021-10-28)

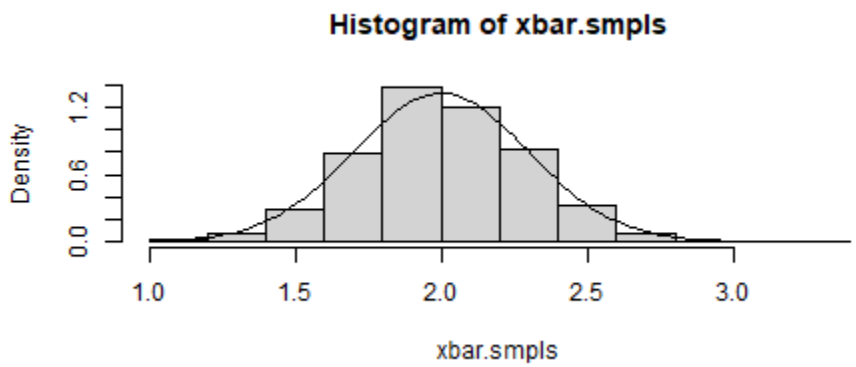
Bootstrap using R

Sampling distribution

- If the population distribution is known, we can sample from it many times to get an idea of the sampling distribution.
- Example: $X_i \sim N(\mu, \sigma^2), i = 1, \ldots, n$.
- Sampling distribution of $\bar{X}$ is $N(\mu, \sigma^2/n)$
- Also, we can draw from normal many times, and plot the histogram of all the means.

Sampling distribution Normal mean

```
set.seed(123)
x = rnorm(100, 2, 3)
xbar.smpls = rep(NA, 1000)
for (i in 1:1000) xbar.smpls[i] = mean(rnorm(100, 2, 3))
hist(xbar.smpls, freq = FALSE)
curve(dnorm(x, mean = 2, sd = 3/10), add = T)
```



Use of sampling distribution

- We can use the sampling distribution to get an idea about the fluctuation of the sample mean etc.
- The true sampling mean should be 2, and true sampling std. dev. should be $\sigma/\sqrt{n} = 3/\sqrt{(100)} = 0.3$.

```
mean(xbar.smpls)
```

```
[1] 2.00255
```

```
sd(xbar.smpls)
```

```
[1] 0.2851543
```

Bootstrap

1. If F is completely unknown, we cannot generate random numbers from this distribution.
2. $X_1, \ldots, X_n$ are the only realizations of F known to us. The idea is to approximate F by $\hat{F}$ based on $X_1, \ldots, X_n$.
3. Basic idea: Inference about a population from sample data, (sample $\rightarrow$ population), can be modeled by resampling the sample data and performing inference about a sample from resampled data, (resampled $\rightarrow$ sample).

Algorithm

1. Data: $X_1, \ldots, X_n$. Test statistics t_n (e.g. $\bar{X}$)
2. Draw B samples $x_1^b, x_2^b, \ldots, x_n^b$ with replacement $b = 1, \ldots, B$ (there might be ties). This is called a **Bootstrap sample**!
3. For each of the B samples calculate the statistic t_n^b.
4. Use these B realizations of t_n to compute whatever you want, e.g.

- mean: $E(t_n) \approx 1/B \sum_b t_n^b$.
- variance: $Var(t_n) \approx 1/(B-1) \sum_b (t_n^b - \bar{t}_n)^2$.
- Empirical CDF $\hat{F}(t_n)$.

Bootstrap in R - the sample() function

A major component of bootstrapping is being able to resample a given data set and in R the function which does this is the sample function.

sample(x, size, replace, prob)

- The first argument is a vector containing the data set to be resampled or the indices of the data to be resampled.
- The size option specifies the sample size with the default being the size of the population being resampled.
- The replace option determines if the sample will be drawn with or without replacement where the default value is FALSE, i.e. without replacement.
- The prob option takes a vector of length equal to the data set given in the first argument containing the probability of selection for each element of x.

More on sampling - 1

- Using sample to generate a permutation of the sequence 1:10

```
sample(10)
```

```
  [1]  4  1  5 10  3  9  8  6  7  2
```

- Bootstrap sample from the same sequence (ties)

```
sample(10, replace = T)
```

```
  [1]  1  9  5  8  1 10  7  5  5  6
```

```
sample(10, replace = T)
```

```
  [1]  5 10 10  1  7  5  6  8  8 10
```

More on sampling - 2

- Boostrap sample from the same sequence with probabilities that favor the numbers 1-5

```
prob1 <- c(rep(0.15, 5), rep(0.05, 5))
prob1
```

```
  [1] 0.15 0.15 0.15 0.15 0.15 0.05 0.05 0.05 0.05 0.05
```

```
sample(10, replace = T, prob = prob1)
```

```
  [1] 4 8 4 3 7 5 5 3 1 7
```

Bootstrap in R

- Here is an R code for getting the bootstrap samples from the same x that we saw before.

```
B = 5000
theta.boot = rep(0, B)
n = length(x)
for (b in 1:B) theta.boot[b] = mean(sample(x, replace = TRUE))
```

Bootstrap mean and sd

- We can use the statistics calculated from the bootstrap distribution to get an idea about the fluctuation of the sample mean etc. when the true F is not known.

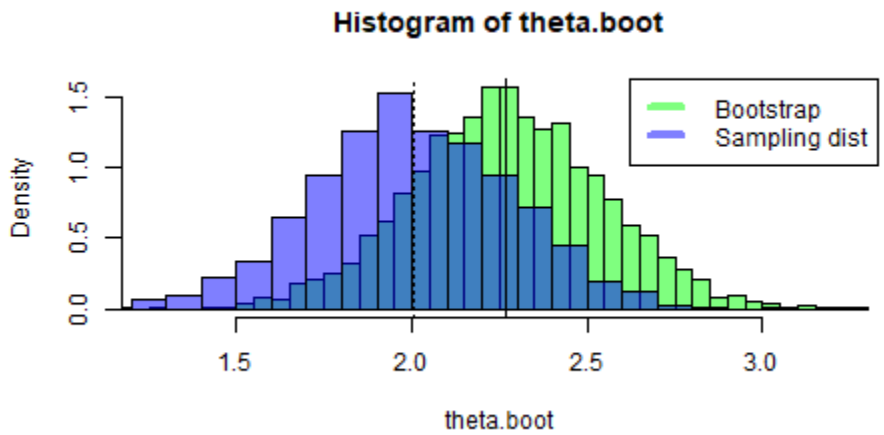
```
mean(theta.boot)
```

```
[1] 2.269501
```

```
sd(theta.boot)
```

```
[1] 0.2727257
```

The bootstrap distribution



Bootstrapping median - 0

- Taken entirely from ATS UCLA website
- In the following bootstrapping example we would like to obtain a standard error for the estimate of the median.
- We will use the `lapply` and `sapply` functions.
- `lapply(X, FUN, ...)`: returns a list of the same length as X, each element of which is the result of applying FUN to the corresponding element of X.
- `sapply(X, FUN, ...)` is a user-friendly version and wrapper of lapply by default returning a vector, matrix or, if `simplify = "array"`, an array if appropriate.

Bootstrapping median - 1

- We want to calculating the standard error of the median for a data set created by taking 100 observations from a normal distribution with mean 5 and stdev 3.
- The distribution of median is not as simple as the mean.

```
data <- rnorm(100, 5, 3)
data[1:10]
```

```
 [1]  6.733251  8.033396 10.952668  2.321523  5.784268  3.614725  5.427864
 [8]  3.917381  2.808604  2.062305
```

Bootstrapping median - 2

- We will generate only 20 bootstrap samples.
- Show the first sample for demonstration.

```
resamples <- lapply(1:20, function(i) sample(data, replace = T))
resamples[1]
```

```
[[1]]
  [1]  6.5408603  1.0506880  2.2906844  5.4154695  0.8284962  5.4623857
  [7]  2.6849414  2.8790922 -1.1825335  6.8980367  2.0623048  0.8284962
 [13]  4.6617780  3.9173813  4.6901455  3.6319773  1.5631956  2.1929835
 [19]  7.2478484  5.3055196  4.9760104  2.8086036  3.1998967  3.8731326
 [25]  7.2868731  6.7378142 10.4292735  8.3667153  6.9539595  1.9988276
 [31]  6.7378142  1.9988276  2.6855018  8.3667153  4.6901455  4.4547279
 [37]  3.9173813  9.8425477  7.9312938  6.3502594  9.0273332  7.2868731
 [43]  5.3055196  5.4623857  2.1125888  2.6855018  0.8284962  0.8284962
 [49]  2.6855018  5.3055196  5.4154695  6.1671975  7.0864048  7.2868731
 [55]  2.0926952  5.6320070  3.8841552  2.5177823  7.7264152  3.8731326
 [61]  1.1129027  1.3204487  6.8980367  7.2868731  6.5395316  3.6319773
 [67]  8.3667153  1.5631956  5.6320070  4.0628588  5.7842685  1.9742257
 [73]  2.2070494  3.0447383  5.0774720  6.9539595  7.9312938  2.8086036
 [79]  2.6849414  5.0774720  5.6856782  1.3344209  7.5653981  1.3204487
 [85]  8.0333956  5.4960551 10.4292735  4.9447316  8.3667153  2.6849414
 [91]  4.6617780 10.4292735  5.5259351  6.5408603  4.4547279  6.5408603
```

Bootstrapping median - 3

- For each bootstrap samples, we will calculate the median.
- Now we have a bootstrap sample from the distribution of the sample median, and we can use it to calculate the standard deviation.

```
r.median <- sapply(resamples, median)
r.median
```

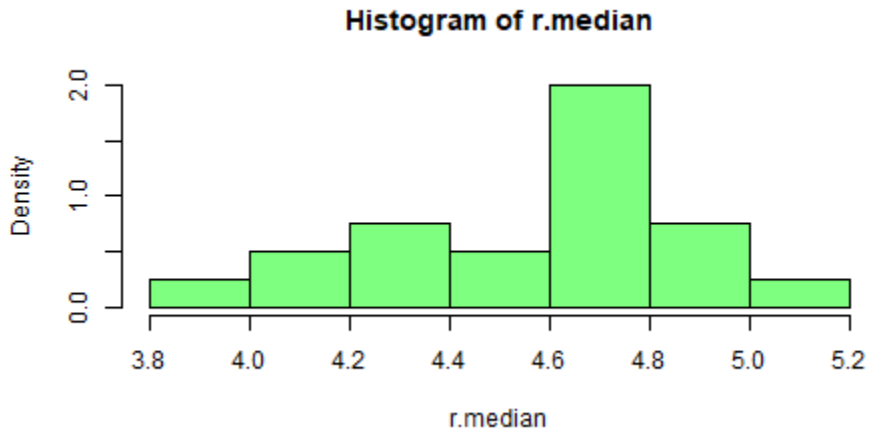
```
  [1] 4.817439 4.718712 4.558253 4.221214 4.791569 4.661778 4.062859 4.52902
  [9] 3.884155 4.632554 4.661778 4.661778 4.799835 4.962863 4.740317 5.01359
 [17] 4.257172 4.363435 4.962863 4.131543
```

```
sqrt(var(r.median))
```

```
 [1] 0.3189351
```

Bootstrapping median - 4

```
hist(r.median, freq = FALSE, col = rgb(0, 1, 0, 0.5))
```



Using package

Bootstrap the 95% CI for R^2 statistic.

First need function to obtain R-Squared from the data

```
rsq <- function(formula, data, indices) {
    d <- data[indices, ]  # allows boot to select sample
    fit <- lm(formula, data = d)
    return(summary(fit)$r.square)
}
```

Botstrapping with 1000 replications:

```
library(boot)
results <- boot(data = mtcars, statistic = rsq, R = 1000, formula = r
    wt + disp)
```

Using package

```r
# view results
results
```

```
 ORDINARY NONPARAMETRIC BOOTSTRAP


 Call:
 boot(data = mtcars, statistic = rsq, R = 1000, formula = mpg ~
     wt + disp)


 Bootstrap Statistics :
      original     bias    std. error
 t1* 0.7809306 0.01148737  0.04958466
```

Using package

You can also get them separately:

```
(bs.median = results$t0)
```

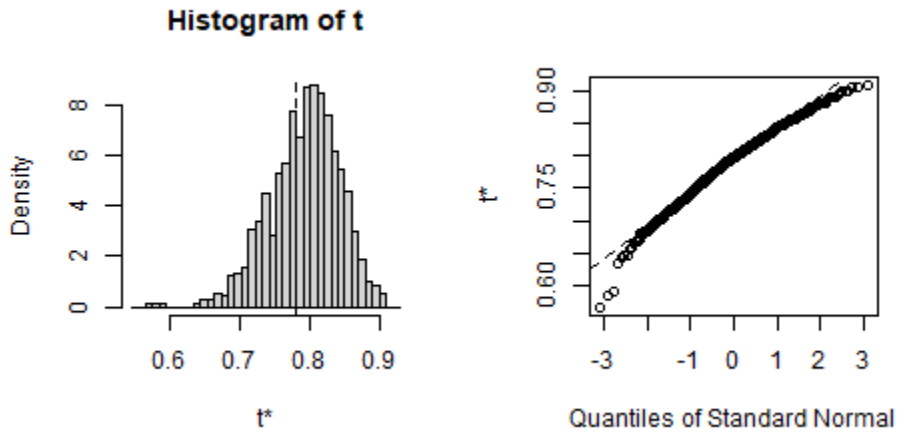
```
 [1] 0.7809306
```

```
(bs.stderr = sqrt(var(results$t)))
```

```
            [,1]
 [1,] 0.04958466
```

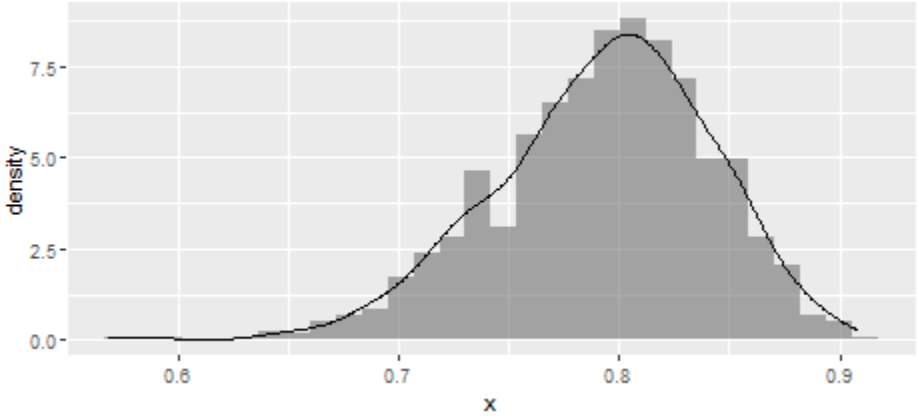
Plotting results

```
plot(results)
```



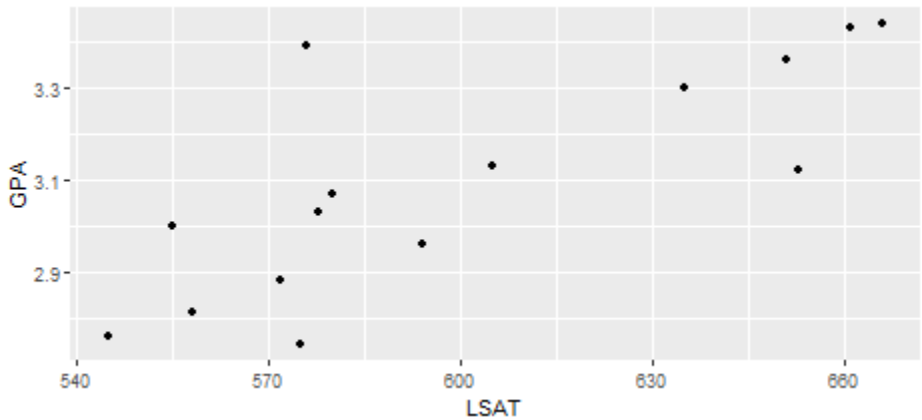
Plotting results

```
df <- data.frame(x = results$t)
x <- df$x
library(ggplot2)
ggplot(df, aes(x)) + geom_histogram(aes(y = ..density..), alpha = 0.5
    geom_density()
```



Law data example

```
setwd("~/Course Notes/stat5525/2021/R codes")
load("law.rda")
library(ggplot2)
qplot(LSAT, GPA, data = law)
```




```
with(law, cor(LSAT, GPA))
```

```
[1] 0.7763745
```

Sample with replacement

```
sample(nrow(law), replace = TRUE)

  [1]  2 15  4  6  3  7  3  2  2 13 13  8  7  2  4

sample(nrow(law), replace = TRUE)

  [1]  7  5  9  7  2  4  8  1 10  8 12 12 12  3  2

with(law[sample(nrow(law), replace = TRUE), ], cor(LSAT, GPA))

 [1] 0.7122137

with(law[sample(nrow(law), replace = TRUE), ], cor(LSAT, GPA))

 [1] 0.6926734

with(law[sample(nrow(law), replace = TRUE), ], cor(LSAT, GPA))

 [1] 0.7429673
```

Bootstrap (Long code)

```
ptm <- proc.time()
B <- 5000
cor.boot <- rep(0, B)
for (b in 1:B) {
    law.boot <- law[sample(nrow(law), replace = TRUE), ]
    cor.boot[b] = cor(law.boot$LSAT, law.boot$GPA)
}
summary(cor.boot)
```

```
    Min. 1st Qu.  Median    Mean 3rd Qu.    Max. 
 0.07721 0.69098 0.79229 0.77206 0.87380 0.99462 
```

```
proc.time() - ptm
```

```
    user  system elapsed 
    0.33    0.00    0.33 
```

Bootstrap (Short code)

```
ptm <- proc.time()
B <- 5000
cors <- replicate(B, {
    with(law[sample(nrow(law), replace = TRUE), ], cor(LSAT,
        GPA))
})
summary(cors)
```

```
     Min.  1st Qu.   Median     Mean  3rd Qu.     Max.
 -0.05628  0.68932  0.78942  0.77030  0.87292  0.99626
```

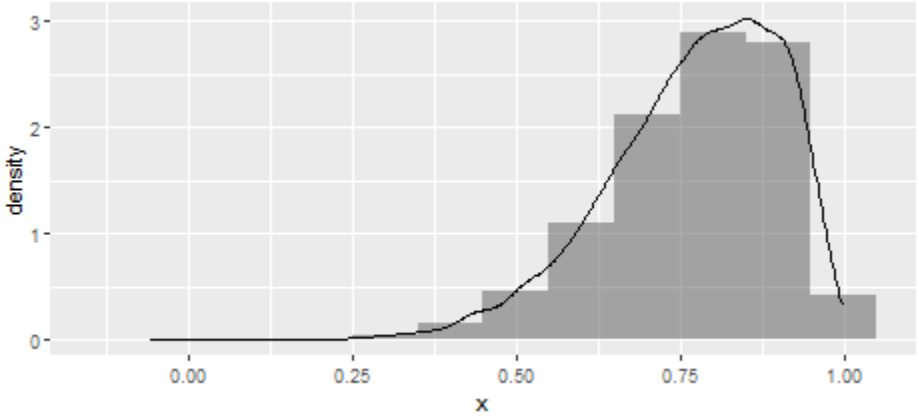
```
proc.time() - ptm
```

```
    user  system elapsed
    0.42    0.00    0.42
```

- Slight gain in running time! Gets better with bigger data.

Plotting results

```
df <- data.frame(x = cors)
x <- df$x
library(ggplot2)
ggplot(df, aes(x)) + geom_histogram(aes(y = ..density..), binwidth =
    alpha = 0.5) + geom_density()
```



How many bootstrap samples do we need?

- We will perform Bootstrap sampling for different values of B, the number of Bootstrap samples and observe the fluctuation between bootstrap estimates for each choice.
- Our preferred B should lead to a low variation.
- We will show the standard loop-y code, which is not efficient and a fast code.

Design

```
Bs <- rep(100 * (1:50), 3)
Bs
```

```
  [1]  100  200  300  400  500  600  700  800  900 1000 1100 1200 1300 1400
 [16] 1600 1700 1800 1900 2000 2100 2200 2300 2400 2500 2600 2700 2800 2900
 [31] 3100 3200 3300 3400 3500 3600 3700 3800 3900 4000 4100 4200 4300 4400
 [46] 4600 4700 4800 4900 5000  100  200  300  400  500  600  700  800  900
 [61] 1100 1200 1300 1400 1500 1600 1700 1800 1900 2000 2100 2200 2300 2400
 [76] 2600 2700 2800 2900 3000 3100 3200 3300 3400 3500 3600 3700 3800 3900
 [91] 4100 4200 4300 4400 4500 4600 4700 4800 4900 5000  100  200  300  400
[106]  600  700  800  900 1000 1100 1200 1300 1400 1500 1600 1700 1800 1900
[121] 2100 2200 2300 2400 2500 2600 2700 2800 2900 3000 3100 3200 3300 3400
[136] 3600 3700 3800 3900 4000 4100 4200 4300 4400 4500 4600 4700 4800 4900
```

How many? (slow code)

```
ptm <- proc.time()
corsize2 = matrix(0, length(Bs),
for (i in 1:length(Bs)) {
    b = Bs[i]
    cor.boot <- numeric(b)
    for (j in 1:b) {
        law.boot <- law[sample(n
        cor.boot[j] = cor(law.bo
    }
    corsize2[i, ] = c(b, mean(co
}
colnames(corsize2) = c("B", "cor
proc.time() - ptm
```

```
    user  system elapsed
   23.77    0.00   23.89
```

```
head(corsize2, n = 6)
```

```
        B       cor
[1,] 100 0.7731894
[2,] 200 0.7626174
[3,] 300 0.7773320
[4,] 400 0.7712480
[5,] 500 0.7673362
[6,] 600 0.7645024
```

Fast code for Bootstrap sample size

- The idea is to calculate the bootstrap correlation for each sample size from 100 to 5000, and replicate three times.
- The slow code uses a nested for loop, which is really inefficient.
- We can use a function called `ldply` that can avoid for loops altogether.
- ldply function : For each element of a list, apply function then combine results into a data frame.
- Our list is values of B, for each B, we will calculate mean of correlation estimates for a bootstrap with B samples.

How many? (Fast Code)

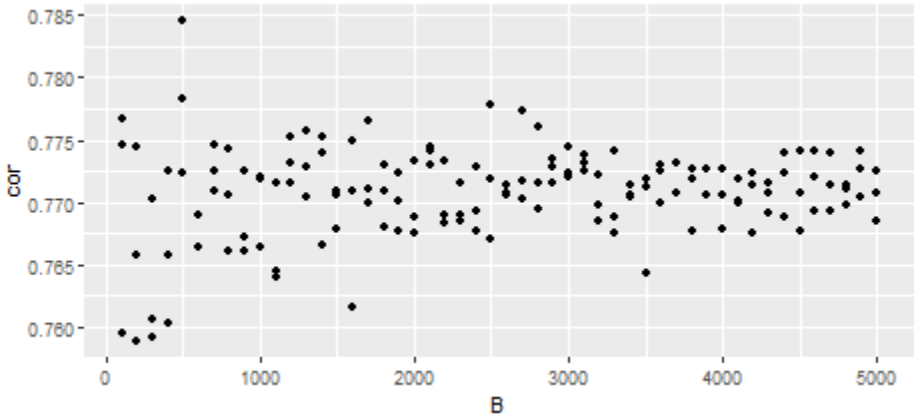
```
ptm <- proc.time()
library(plyr)
corsize <- ldply(Bs, function(B) {
    cors <- replicate(B, {
        with(law[sample(nrow(law), replace = TRUE), ], cor(LSAT,
            GPA))
    })
    c(B = B, cor = mean(cors))
})
head(corsize, n = 3)
```

```
     B       cor
 1 100 0.7745522
 2 200 0.7743736
 3 300 0.7702288
```

```
proc.time() - ptm
```

```
    user  system elapsed
   26.86    0.00   27.02
```

Plot

- Plot the correlation estimates for each value of B for three choices.
- As B increases, fluctuation diminishes.

```
qplot(B, cor, data = corsize)
```